

DOCUMENT 04

System Architecture & Design

Technology stack, component map, data model, design patterns, API design, and Architecture Decision Records (ADRs).

SCOPE Stack • Components • Data Model • Patterns • APIs • ADRs

SPONSOR Ghana Chamber of Gold Buyers

TECHNOLOGY PARTNER Vanta Technologies Ltd.

CONFIDENTIAL • CHAMBER & VENDOR INTERNAL • AUTHORIZATION DATE 15 AUGUST 2026

TABLE OF CONTENTS

Document 4 — System Architecture & Design..... 3

 4.1 Technology Stack..... 3

 4.2 High-Level Component Map..... 4

 4.3 Data Model — Core Entities..... 4

 4.4 Design Patterns..... 5

 4.5 API Design..... 6

 Conventions..... 6

 Authentication..... 6

 Key Endpoints (Selected)..... 6

 4.6 Architecture Decision Records (ADRs)..... 7

 ADR-001 — Adopt Event-Driven SOA over Monolith..... 7

 ADR-002 — Use Auth0 B2B for Identity..... 7

 ADR-003 — Use Kafka (AWS MSK) over RabbitMQ/SQS..... 8

 ADR-004 — Use the Outbox Pattern for Atomic Event Publish..... 8

 ADR-005 — Next.js PWA, No Native Apps in Phases 1–4..... 8

Right-click the table of contents and select “Update Field” to refresh page numbers after opening in Microsoft Word.

Document 4 — System Architecture & Design

The AurumX platform is built around six architecture principles: member-gated (not open marketplace); event-driven core; audit-first data model; vendor and partner agnostic; mobile-first delivery with desktop-required for high-value decisions; cost-aware scaling. Each principle is tied to a Charter constraint or risk.

4.1 Technology Stack

LAYER	TECHNOLOGY	RATIONALE
Frontend (web/PWA)	Next.js 16 + React 19 + TypeScript + Tailwind CSS	Server-rendered PWA supports low-bandwidth mobile use; strong typing for safety-critical financial UX
Backend framework	NestJS (Node.js 22 LTS, TypeScript)	Opinionated module system; first-class DI; matches frontend language; strong OpenAPI generation
API style	REST (primary) + GraphQL (BFF) + Webhooks + WebSocket	REST for partner integrations; GraphQL for portal; WebSocket for negotiation latency
Primary database	PostgreSQL 16	ACID guarantees for trade ledger; JSONB for flexible metadata; Row-Level Security for tenant isolation
Cache	Redis 7 (cluster mode)	Session store, rate-limit counters, real-time negotiation state
Search	Elasticsearch 8	Lot discovery, full-text on member directory, audit-log search
Event bus	Apache Kafka (AWS MSK)	Domain events for audit log, anomaly detection, notification fan-out; 7-year retention
Object storage	AWS S3 + KMS encryption	Document storage with lifecycle to Glacier for archival

LAYER	TECHNOLOGY	RATIONALE
Identity	Auth0 (B2B tenant)	Enterprise agreement in place; supports organization-level RBAC
Container orchestration	AWS EKS (managed Kubernetes)	Multi-AZ, autoscaling, mature ecosystem
CI/CD	GitHub Actions + Argo CD (GitOps)	Branch-driven builds; declarative K8s deployments
Observability	Datadog (APM + logs + metrics) + Sentry	Unified observability; strong OpenTelemetry support
IaC	Terraform + Helm	Cloud-agnostic; Helm for K8s release management
Cloud	AWS (af-west-1 primary, eu-west-1 DR)	African region for Ghana data residency; EU region for DR

4.2 High-Level Component Map

The platform is organized into five tiers: Client tier (PWA + Chamber Admin Console); Edge tier (CloudFront + WAF); API tier (Kong Gateway + Auth0); Service tier (EKS-hosted NestJS services with Integration Adapters and Background Workers); Data tier (PostgreSQL, Redis, Kafka, Elasticsearch, S3). All stateful services span three Availability Zones with synchronous replication for RDS. Partner integrations (banks, vaults, assay labs, government APIs) are abstracted behind Strategy-pattern interfaces so one provider can be swapped for another — directly mitigating Charter Risk R2 (escrow partner withdrawal).

4.3 Data Model — Core Entities

The data model is organized around six core aggregates: Organization (members with tier, license, KYC status); GoldLot (weight, purity, serial number, provenance chain); RFQ/Auction + Bid; Trade + Escrow + Contract; TradeEvent (immutable audit); ComplianceAlert + AlertEvent. Each aggregate has a defined state machine (see Document 5).

AGGREGATE	KEY FIELDS	OWNING SERVICE
Organization	id, name, tier (TIER_1/TIER_2/SELLER), status, kyc_status	Member Service
GoldLot	id, seller_org_id, weight_grams, purity, serial_number, status	Lot Service

AGGREGATE	KEY FIELDS	OWNING SERVICE
RFQ / Auction	id, lot_id, buyer_org_id, asking_price_usd, status	RFQ Engine / Auction Engine
Bid	id, rfq_id, bidder_org_id, amount_usd, status	RFQ / Auction Engine
Trade	id, lot_id, buyer_org_id, seller_org_id, amount_usd, status, settled_at	Trade Service
Escrow	id, trade_id, bank_account_id, amount_usd, status, bank_reference	Escrow Service
TradeEvent	id, trade_id, event_type, event_payload, hash_anchor, occurred_at	Audit Log Service
ComplianceAlert	id, triggered_by_trade_id, severity, status, signals	Compliance Service

4.4 Design Patterns

PATTERN	WHERE APPLIED	WHY
Domain-Driven Design (DDD)	Core domain services	Complex business rules require rich domain models, not anemic CRUD
Event Sourcing (partial)	Trade, Compliance Alert aggregates	Audit-defensible history; replayable for investigations
CQRS	Lot read model (Elasticsearch); write model (PostgreSQL)	Optimization for high-read lot discovery
Repository + Unit of Work	All data access layers	Testability; consistency boundaries per aggregate
Strategy	EscrowProvider, PaymentProvider, NotificationProvider, AssayProvider	Vendor-agnostic — provider swap mitigates Charter Risk R2
Outbox Pattern	All services publishing to Kafka	Guarantees atomic DB write + event publish (no dual-write inconsistency)
Saga (choreography)	Trade lifecycle (8+ stages)	Long-running distributed transaction without 2PC

PATTERN	WHERE APPLIED	WHY
Materialized View	Analytics rollups for compliance dashboard	Decouples heavy read queries from trade-write hot path

4.5 API Design

CONVENTIONS

- HTTPS only (TLS 1.3 minimum). JSON format with ISO 8601 UTC timestamps and decimal-string monetary amounts.
- URL versioning (/v1/, /v2/). Backward-compatible changes within major version; breaking changes require new major with 12-month overlap.
- Cursor-based pagination. Idempotency-Key header required for all POST that creates resources or triggers side effects (24-hour idempotency window).
- Rate limiting: token-bucket per API key. Errors use RFC 7807 Problem Details format.

AUTHENTICATION

Auth0 B2B with Organizations — one Auth0 Organization per Chamber member Organization. Authorization Code + PKCE for the PWA; Client Credentials for machine-to-machine (Tier 1 ERP integration, government API adapters). JWT (RS256), 1-hour access tokens; refresh tokens 30-day sliding window. RBAC model: roles scoped per Organization (org.admin, org.trader, org.compliance, org.viewer) and globally (chamber.compliance, chamber.executive, chamber.ops). MFA required for org.admin, all Chamber roles, and any trade action above US\$100K.

KEY ENDPOINTS (SELECTED)

ENDPOINT	PURPOSE	BACKS USER FLOW
POST /v1/organizations	Create new member organization	UF-01 Onboarding
POST /v1/organizations/{id}/kyc/submit	Submit KYC form	UF-01 Onboarding
POST /v1/lots	Create a new gold lot	UF-02 Lot Creation
POST /v1/lots/{id}/publish	Publish lot to qualified buyers	UF-02 Lot Creation
POST /v1/rfqs	Submit RFQ with asking price	UF-03 RFQ Submission
POST /v1/rfqs/{id}/bids	Submit a bid	UF-03 RFQ Submission
POST /v1/rfqs/{id}/negotiate	Submit counter-offer	UF-03 RFQ Submission

ENDPOINT	PURPOSE	BACKS USER FLOW
POST /v1/rfqs/{id}/accept	Accept terms; trade created	UF-03 RFQ Submission
POST /v1/trades/{id}/escrow/fund	Trigger bank escrow funding	UF-05 Escrow Funding
POST /v1/trades/{id}/escrow/release	Release escrow to seller	UF-05 Escrow Funding
POST /v1/trades/{id}/freeze	Freeze trade (compliance only)	UF-08 Compliance Investigation
GET /v1/compliance/alerts	List compliance alerts	UF-08 Compliance Investigation
POST /v1/compliance/alerts/{id}/events	Acknowledge / escalate / file SAR	UF-08 Compliance Investigation
POST /v1/exports	Initiate export documentation workflow	UF-07 Export Documentation

4.6 Architecture Decision Records (ADRs)

ADR-001 — ADOPT EVENT-DRIVEN SOA OVER MONOLITH

Context: The platform spans heterogeneous concerns (member onboarding, trade lifecycle, escrow, compliance) with sharply different scaling profiles. Compliance requires a 24-hour detection-to-action SLA. SOC 2 Type II is mandatory.

Decision: Event-driven service-oriented architecture (not full microservices) with Kafka as the event backbone, the Outbox pattern for atomic DB writes + event publish, and Saga choreography for the trade lifecycle. One PostgreSQL cluster shared across services via separate schemas; no cross-service direct DB access.

Consequences: (+) Independent scaling; (+) intrinsic audit trail; (+) compliance can subscribe to all trade events without coupling. (–) Operational complexity; (–) distributed-systems pitfalls (eventual consistency, idempotency).

ADR-002 — USE AUTH0 B2B FOR IDENTITY

Context: Platform requires organization-scoped RBAC, MFA, SSO for Tier 1 buyers (SAML), per-organization IP allowlisting. Charter §8 C2 mandates Auth0.

Decision: Auth0 B2B with Organizations. One Auth0 Organization per AurumX Organization. Custom claims for fine-grained permissions. SAML/OIDC enterprise connections for Tier 1 buyers; Auth0-hosted login branded for the Chamber.

Consequences: (+) SOC 2 already in place for Auth0; (+) MFA and SSO built-in. (-) Vendor lock-in; (-) per-MAU pricing can scale steeply.

ADR-003 — USE KAFKA (AWS MSK) OVER RABBITMQ/SQS

Context: Trade lifecycle is a long-running saga with 8+ stages. Compliance must replay any trade's full event history on demand for investigation and SOC 2 audit. Throughput forecast: ~500 events/sec peak, ~50 events/sec average — modest, but replayability is critical.

Decision: Apache Kafka (AWS MSK Standard, 3-broker multi-AZ) as the event backbone. Outbox pattern for atomic DB write + event publish. Log-compacted audit-log topic as canonical replay source. Retention: 7 years.

Consequences: (+) Replayable event history — direct enabler for compliance; (+) multiple consumers without coupling. (-) MSK operational cost (~US\$1,800/month baseline).

ADR-004 — USE THE OUTBOX PATTERN FOR ATOMIC EVENT PUBLISH

Context: With event-driven architecture (ADR-001) and Kafka (ADR-003), the dual-write problem arises: if a service writes to PostgreSQL and then publishes to Kafka, a failure between the two leaves the system inconsistent. For an audited financial platform, this is unacceptable.

Decision: Every service that publishes events uses the Outbox pattern — in a single DB transaction, write the domain aggregate AND insert a row into the service's local outbox table; a separate worker reads the outbox and publishes to Kafka. Idempotent consumers deduplicate by event ID.

Consequences: (+) Atomicity guarantee between DB state and event stream; (+) recoverable from any failure. (-) Adds an outbox table per service; (-) slight write-latency increase.

ADR-005 — NEXT.JS PWA, NO NATIVE APPS IN PHASES 1-4

Context: Persona coverage spans Kwame (desktop-required), Abena (mobile-first), and Ibrahim (mobile-only on 3G). Native iOS+Android require 2 separate codebases and App Store review cycles.

Decision: Build entire member-facing UI as a Next.js PWA — server-rendered pages for first-paint performance on low-bandwidth connections; service worker for offline-tolerant document capture; responsive design with mobile-first patterns. Push notifications via Web Push API with SMS/WhatsApp fallback.

Consequences: (+) Single codebase across desktop, tablet, mobile; (+) no App Store review cycles. (-) Some Tier 1 buyers may push back; mitigated by excellent PWA experience and API access for SAP integration.